Terraform State Files, HCL Block Syntax & Resource References

Your notes, cleaned up from dictation + screenshots — Lecture 2

Quick recall

Two things tie together in this session:

1. `terraform.tfstate` — the file Terraform uses to track what it created, so it can compare *desired* vs *actual* infrastructure on every run.
2. **HCL block syntax** — the declarative `block_type "block_label" "block_name" { }` shape every `.tf` file is built from, and how one resource **references** another resource's attributes to create implicit dependencies (visualized with `terraform graph`).

---

1. What is the State File?

> The state of a Terraform project is a file that stores all details about resources that were created in the context of a given project.

- Stored by default as a local file named `terraform.tfstate` (JSON format).
- Contains **all information about the provisioned infrastructure that Terraform manages** — both the attributes you gave it *and* the ones Terraform/AWS filled in automatically (public/private IP, availability zone, default security group, volume IDs, etc.).
- Created and updated every time you run `terraform apply` or `terraform destroy`.
- **"It basically acts as a database for provisioned resources."** Terraform uses it to:
  - map real-world resources back to your `.tf` configuration,
  - keep track of resource metadata,
  - improve performance for large infrastructures (so it doesn't have to query the cloud API for every resource on every run).
- Before any operation, Terraform does a **refresh** — it re-checks the real infrastructure and syncs the state file with what's actually there.

Desired State vs Current State

	Definition
Desired State	The state *mentioned in your resource block* of the `.tf` file — i.e. what the infra *should* look like according to your code.
Current State	The *present* state of the resources in the actual infrastructure. Can differ from desired state (manual changes, drift, etc.).

The reconciliation logic (`terraform plan` / `apply`)

```
Tf config files  →  tf state file  →  Target Environment
```

Terraform first looks at what you **want** (desired state, from your `.tf` files), then scans what's **actually** in your cloud platform (current/actual state), and calculates a simple delta:

- In desired but not actual → **it gets created**
- In both, but configured differently → **it gets updated**
- In actual but not desired → **it gets deleted**

> **In short: Terraform is a state-based engine.**

State file safety net: the `.backup` file

Any time Terraform is about to **modify** the state file (add, update, delete), it first backs up the current state file *before* writing the new one — automatically, no extra command needed. This creates `terraform.tfstate.backup` right alongside `terraform.tfstate`.

Inspecting state

```
terraform state list                    # every resource tracked in this state file
terraform state show <resource_addr>    # full attributes of one resource, e.g.:
terraform state show aws_s3_bucket.first
```

---

2. HCL Block Syntax (how a `.tf` file is actually written)

Terraform code lives in `.tf` files, written in **HCL** (HashiCorp Configuration Language). HCL is a **declarative** language — you describe the end state you *want*, not the imperative steps to get there.

The block shape

```
<block_type> "<block_label>" "<block_name>" {
  # Configuration settings for the resource
  attribute1 = value1
  attribute2 = value2
}
```

> **A block in Terraform is like a section of code that tells Terraform what to do.**

Part	What it is	Examples
Block type	Reserved keyword — the *kind* of block	`resource` , `provider` , `variable` , `output` , `module`
Block label	Depends on the provider + resource type	AWS: `aws_instance` , `aws_s3_bucket` , `aws_vpc` , `aws_security_group` · Azure: `azurerm_virtual_machine`
Block name	Your own reference name — a unique identifier for *this* block	`linux` , `first` , `allow_ssh`

Inside the `{ }` you write **attributes** — key/value pairs. Some are **mandatory** (resource won't create without them), others **optional** (Terraform/the provider fills in a default, recorded into the state file once the resource exists — e.g. AZ, public/private IP, default SG).

Worked example — two resource blocks in one `main.tf`

```
########## Launch Ec2 Instance in Mumbai region
resource "aws_instance" "linux" {
  ami           = "ami-0cca134ec43cf708f"
  instance_type = "t2.micro"
  key_name      = "ram_aws"
  tags = {
    Name   = "Test_server"
    source = "terraform"
  }
}

########## Launch S3 Bucket in Mumbai region
resource "aws_s3_bucket" "first" {
  bucket = "linux23212google123"
  tags = {
    Name = "my_buck"
  }
}
```

One `.tf` file can hold **multiple resource blocks** — each independent, addressed as `block_type.block_label.block_name` (e.g. `aws_instance.linux`, `aws_s3_bucket.first`).

What happened when this got applied:

```
[root@Devops-server project1]# terraform state list
aws_instance.linux
aws_s3_bucket.first
```

Only `main.tf` was written by hand — `terraform.tfstate` was **generated automatically** during `init` / `apply`, tracking both resources.

Re-running `terraform plan` with no code changes:

```
aws_s3_bucket.first: Refreshing state... [id=linux23212google123]
aws_instance.linux: Refreshing state... [id=i-0360dc4b6f1fe0859]

No changes. Your infrastructure matches the configuration.
```

Desired state (code) == current state (real infra) == state file → nothing to do.

Editing an attribute (adding a tag) and re-planning:

```
Terraform will perform the following actions:

  # aws_s3_bucket.first will be updated in-place
  ~ resource "aws_s3_bucket" "first" {
        id   = "linux23212google123"
      ~ tags = {
            "Name" = "my_buck"
          + "Team" = "Linux"
        }
      ~ tags_all = {
          + "Team" = "Linux"
            # (1 unchanged element hidden)
        }
        # (14 unchanged attributes hidden)
        # (3 unchanged blocks hidden)
    }

Plan: 0 to add, 1 to change, 0 to destroy.
```

Only the **delta** shows. After `apply`, `terraform.tfstate.backup` appears — a backup of the state *before* this change was applied.

---

3. `terraform graph` — visualizing dependencies

> Generates a visual representation of your infrastructure's configuration or execution plan. Maps out resource dependencies using the machine-readable **DOT** language. This text output can be passed to a visualization tool like **Graphviz** to create an architecture diagram.

```
yum install graphviz                        # provides the `dot` renderer
terraform graph -type=plan | dot -Tpng > graph.png
```

- `terraform graph` alone emits DOT-format text describing the dependency graph.
- Piping through `dot -Tpng` (Graphviz) renders that text into an actual `.png` image.
- Read the resulting graph **bottom-up**: resources at the bottom (only depending on the provider) get created first; resources above them depend on what's below.

---

4. Resource references (how one resource "finds" another)

Reference syntax:

```
<block_label>.<block_name>.<attribute>
```

Once you've written a security group block, you can reference its generated `id` from **any other block** as `aws_security_group.allow_ssh.id` — even though that security group doesn't exist yet when you write the code. Terraform resolves the actual value at apply time and, critically, uses this reference to figure out **creation order automatically** (implicit dependency — no `depends_on` needed).

Worked example — SG → ingress rule

```
resource "aws_security_group" "allow_ssh" {
  name        = "my_sg"
  description = "Allow ssh traffic and all outbound traffic"
  tags = {
    Name = "allow_ssh"
  }
}

resource "aws_vpc_security_group_ingress_rule" "allow_ssh_ipv4" {
  security_group_id = aws_security_group.allow_ssh.id
  cidr_ipv4         = aws_vpc.main.cidr_block
  from_port         = 443
  ip_protocol       = "tcp"
  to_port            = 443
}
```

```
Plan: 2 to add, 0 to change, 0 to destroy.
aws_security_group.allow_ssh: Creating...
aws_security_group.allow_ssh: Creation complete after 3s [id=sg-04b12c00bff517016]
aws_vpc_security_group_ingress_rule.allow_ssh_ipv4: Creating...
aws_vpc_security_group_ingress_rule.allow_ssh_ipv4: Creation complete after 0s [id=sgr-0ae57
```

The security group **must** exist before the ingress rule can reference its `id` — Terraform enforces that automatically.

Worked example — attaching that SG to an EC2 instance

```
resource "aws_instance" "linux" {
  ami                    = "ami-0cca134ec43cf708f"
  instance_type          = "t2.micro"
  key_name               = "deployer-key"
  vpc_security_group_ids = [aws_security_group.allow_ssh.id]
  tags = {
    Name   = "Test_server"
    source = "terraform"
  }
}
```

> **HCL gotcha:** `vpc_security_group_ids` expects a **list**, so the reference must be wrapped in square brackets — `[aws_security_group.allow_ssh.id]` — even for a single value. Without the brackets it's the wrong type for that attribute.

Worked example — EBS volume + attachment referencing the instance

```
resource "aws_volume_attachment" "ebs_att" {
  device_name = "/dev/sdc"
  volume_id   = aws_ebs_volume.example.id
  instance_id = aws_instance.linux.id
}

resource "aws_ebs_volume" "example" {
  availability_zone = "ap-south-1b"
  size              = 5
}
```

```
Plan: 2 to add, 0 to change, 0 to destroy.
aws_ebs_volume.example: Creating...
aws_ebs_volume.example: Still creating... [00m10s elapsed]
aws_ebs_volume.example: Creation complete after 11s [id=vol-01555d00742df9baa]
aws_volume_attachment.ebs_att: Creating...
aws_volume_attachment.ebs_att: Still creating... [00m10s elapsed]
```

Again the order is automatic: the **volume** must exist before the **attachment** can reference its `id`, and the **instance** must exist before the attachment can reference *its* `id`.

Final dependency graph

Built up: **key pair** → **security group** → **EBS volume** → **EC2 instance** → **volume attachment** → **ingress rule**, read bottom-to-top:



The graph visually proves what the reference syntax already implied: Terraform figured out the whole build order from `resource_x.resource_y.id`-style references, with **zero** manual ordering or `depends_on`.

CLI additions from this session

Command	What it does
`terraform state list`	List every resource currently tracked in state
`terraform state show <resource>`	Show full attributes of one resource from state
`terraform graph`	Emit the dependency graph in DOT format
`terraform graph -type=plan | dot -Tpng > graph.png`	Render the dependency graph to a PNG (needs `graphviz`)
`yum install graphviz`	Install the Graphviz renderer (`dot`) used above

Bonus — more commands referenced this session (not yet hands-on, good to know)

Command	What it does
`terraform taint`	Manually marks a specific resource as **tainted** — treated as damaged/needing replacement on next `apply`
`terraform import`	Brings **existing infrastructure** (created outside Terraform) under Terraform's management
`terraform workspace`	Manage multiple **deployments of the same configuration** (e.g. dev/stage/prod) via separate state per workspace
`terraform refresh`	Reads current settings from all managed remote objects and updates state to match — the same refresh that happens implicitly before `plan` / `apply`
(logging)	Terraform logging records the **internal steps** Terraform takes while managing infrastructure
(provisioners)	Tools used within a config to **execute scripts/commands** on a local or remote machine as part of provisioning
(plugin cache)	Local storage that saves provider plugins so `init` is faster and more reliable
(user data)	Instructions given to a new server (e.g. EC2) to run automatically at startup

Practice Tasks

- ☐ Apply a config with two independent resources (e.g. EC2 + S3), then confirm `terraform.tfstate` is auto-created and both show up in `terraform state list`.

☐ Run `terraform plan` twice in a row with no code changes and confirm "No changes. Your infrastructure matches the configuration."

☐ Add one new tag to an existing resource's `tags` block, run `plan`, and read the `~ update in-place` diff carefully — confirm only the delta is shown.

☐ Apply that tag change and confirm `terraform.tfstate.backup` appears automatically, with no separate command.

☐ Write a security group + an ingress rule as two separate resource blocks, reference the SG's `.id` in the ingress rule, and watch the apply log to confirm the SG is created *before* the ingress rule.

☐ Attach that security group to an EC2 instance via `vpc_security_group_ids = [aws_security_group.<name>.id]` — deliberately drop the `[ ]` once to see the type error HCL throws.

☐ Create an EBS volume and an `aws_volume_attachment` resource referencing both the volume's and the instance's `.id`, and confirm creation order in the apply log.

☐ Install `graphviz`, run `terraform graph -type=plan | dot -Tpng > graph.png`, and open the PNG — trace the dependency chain bottom-up and match it to the reference chain you wrote in code.

What I practiced

- Understood what the state file is, why it exists ("database for provisioned resources"), and how it maps real-world resources back to `.tf` config.
- Learned the Desired State vs Current State distinction and the create/update/delete delta logic Terraform uses on every `plan` / `apply`.
- Confirmed the automatic `terraform.tfstate.backup` behavior before any state-modifying operation.
- Practiced `terraform state list` and `terraform state show <resource>`.
- Learned the `block_type "block_label" "block_name" { attributes }` HCL shape and wrote a two-resource `main.tf` (EC2 instance + S3 bucket) from scratch.
- Wrote implicit resource references (`aws_security_group.allow_ssh.id`) across a security group → ingress rule → EC2 instance → EBS volume/attachment chain, and watched Terraform resolve creation order automatically — no `depends_on` needed.
- Installed `graphviz` and generated a real dependency graph PNG with `terraform graph -type=plan | dot -Tpng > graph.png`, matching it against the reference chain in code.

Merged into the master reference notes: terraform-master-notes.md — "2. Terraform State Files, HCL Block Syntax & Resource References" section.
Screenshots for this session: ../Screenshots/02-terraform-state-and-resource-references-screenshots.html (25 shots)